



UNINASSAU

Jogo Mario

Disciplina: Front-End Frameworks

Professor: João Ferreira da Silva Júnior

ADRIENNE ELLEN SALGADO - SCRUM MASTER - 01794945

EVELYN ALINE NUNES MINEO - DOCUMENTADOR - 01824704

MATEUS ENRIQUE - DESENVOLVEDOR INTERFACE - 01846550

MARCUS VINICIOS SOUZA DE PABLO - DOCUMENTADOR - 01846641

FLAVIO JOSÉ REGO DA SILVA JÚNIOR - DESENVOLVEDOR LÓGICA - 01790691

ROBERTO SALGADO DA SILVA JÚNIOR - TESTADOR - 01821769

Olinda

Setembro, 2026

1. DESCRIÇÃO DO PROJETO

O projeto consiste no desenvolvimento de um jogo de navegador inspirado na dinâmica do jogo do dinossauro do Chrome. Na partida, o jogador controla um personagem que deve pular para evitar canos que se deslocam pela tela.

O jogo apresenta mecânicas de movimentação, animação, detecção de colisões, encerramento da partida e reinicialização. Seu desenvolvimento utiliza HTML5, CSS3 e JavaScript puro, permitindo a execução diretamente em um navegador.

2. OBJETIVO DO JOGO

O objetivo do jogo é fazer com que o personagem evite os canos pelo maior tempo possível, utilizando o comando de pulo para desviar dos obstáculos. Durante a partida, os canos se movimentam pela tela e o jogador deve reagir para evitar colisões. Caso o personagem colida com um cano, o jogo é encerrado e a mensagem “Game Over” é exibida.

O jogador pode iniciar uma nova partida por meio do botão “Reiniciar”. Está prevista também a implementação de um sistema de pontuação para acompanhar o desempenho do jogador.

3. REQUISITOS DO JOGO

Os requisitos do projeto descrevem as funcionalidades e características esperadas do jogo. Eles servem como base para orientar o desenvolvimento e verificar se o sistema atende à proposta estabelecida.

3.1 REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem as ações e funcionalidades que o jogo deve realizar.

- RF01** — O jogador deve poder fazer o Mario pular apertando a barra de espaço.
- RF02** — O sistema deve mover o cano continuamente da direita para a esquerda da tela.
- RF03** — O sistema deve reposicionar o cano no lado direito assim que ele sair da tela.
- RF04** — O sistema deve detectar colisão entre o Mario e o cano.
- RF05** — O sistema deve aumentar gradualmente a velocidade do cano com o tempo de jogo.
- RF06** — Ao colidir com o cano, o sistema deve exibir a mensagem “Game Over” e interromper o jogo.
- RF07** — Ao colidir, o Mario deve executar uma animação de queda.
- RF08** — O jogador deve poder reiniciar o jogo pelo botão “Reiniciar”.
- RF09** — Ao reiniciar, todas as variáveis do jogo devem voltar ao estado inicial.
- RF10** — O sistema deve contabilizar e exibir a pontuação do jogador durante a partida.

3.2 REQUISITOS NÃO FUNCIONAIS

- RNF01** — O jogo deve ser executado inteiramente em um navegador, sem necessidade de backend.
- RNF02** — O jogo deve ser desenvolvido utilizando HTML5, CSS3 e JavaScript puro.
- RNF03** — O layout deve ser responsivo, permitindo a utilização em telas menores.

4. TECNOLOGIAS E RECURSOS UTILIZADOS

O projeto utiliza tecnologias web para a construção da interface, estilização e implementação da lógica do jogo.

HTML5: utilizado para estruturar os elementos da página e do jogo.

CSS3: utilizado para estilizar os elementos e organizar a apresentação visual da interface.

JavaScript: utilizado para implementar a lógica, as interações, a movimentação e as regras do jogo.
requestAnimationFrame(): utilizado para realizar a atualização contínua das animações do jogo.
getBoundingClientRect(): utilizado para obter a posição e as dimensões dos elementos, auxiliando na detecção de colisões.

5. FUNCIONAMENTO DO JOGO

O jogo apresenta um personagem que deve desviar de canos que se movimentam continuamente da direita para a esquerda da tela. O jogador utiliza a barra de espaço para fazer o Mario pular. Conforme o cano sai da área visível, ele é reposicionado no lado direito da tela, permitindo a continuidade dos obstáculos durante a partida. O sistema realiza a detecção de colisões entre o personagem e o cano. Para isso, é utilizado o recurso `getBoundingClientRect()`, que permite obter informações sobre a posição e as dimensões dos elementos na tela. Quando ocorre uma colisão, o jogo exibe a mensagem “Game Over”, interrompe a partida e executa uma animação de queda do personagem. A animação e a atualização contínua do jogo utilizam o recurso `requestAnimationFrame()`, responsável por auxiliar na execução das atualizações visuais no navegador. O jogador pode utilizar o botão “Reiniciar” para iniciar uma nova partida, fazendo com que as variáveis do jogo retornem ao estado inicial. O sistema de pontuação será adicionado posteriormente ao projeto.

6. INSTRUÇÕES DE INSTALAÇÃO E EXECUÇÃO

6.1 PRÉ-REQUISITOS

Para executar o projeto, são necessários:

- Um navegador web atualizado.
- Git, caso o usuário opte por clonar o repositório.
- Acesso ao repositório do projeto.

6.2 INSTALAÇÃO

1. Acesse o repositório do projeto no GitHub.
2. Copie o link do repositório.
3. Abra um terminal ou Git Bash.
4. Execute o comando:
Git clone <https://github.com/EllenSalgado2028/jogoMario.git>
5. Acesse a pasta do projeto após a clonagem.

6.3 EXECUÇÃO

1. Abra a pasta do projeto.
2. Localize o arquivo `index.html`.
3. Abra o arquivo em um navegador web.
4. Inicie a partida.
5. Utilize a barra de espaço para fazer o personagem pular.
6. Caso ocorra uma colisão, clique no botão “Reiniciar” para iniciar uma nova partida.

7. GIT E GITHUB

O Git e o GitHub foram utilizados para o controle de versão, o compartilhamento dos arquivos e a organização do desenvolvimento do projeto. O Front-End foi desenvolvido na branch dev, permitindo que as alterações fossem realizadas e registradas de maneira organizada por meio de commits. Após as modificações, os arquivos foram enviados ao repositório remoto utilizando o comando git push. Durante o desenvolvimento, foram utilizados os seguintes comandos:

git status: verifica o estado atual dos arquivos do repositório, identificando alterações, arquivos adicionados ou pendentes.

git add: adiciona as alterações à área de preparação para que possam ser incluídas no próximo commit.

git commit: registra as alterações realizadas no projeto, acompanhadas de uma mensagem descritiva.

git push: envia os commits locais para o repositório remoto no GitHub.

git branch: lista as branches existentes ou permite o gerenciamento delas.

git branch -a: exibe as branches locais e remotas disponíveis no repositório.

git switch -c dev: cria uma nova branch chamada dev e muda para ela.

git log -1: consulta e exibe o último commit realizado, permitindo verificar o registro da etapa mais recente do desenvolvimento.

git pull: baixa as alterações do repositório remoto e atualiza o repositório local, sincronizando os arquivos e commits mais recentes do GitHub.

Esses comandos contribuíram para manter o histórico de alterações organizado, facilitar o acompanhamento do desenvolvimento e garantir o armazenamento do projeto no repositório remoto.

8. CONSIDERAÇÕES FINAIS

A documentação apresenta a proposta, o objetivo, os requisitos, as tecnologias e o funcionamento do jogo desenvolvido.

O projeto busca aplicar conceitos de desenvolvimento web, animação, interação com o usuário e detecção de colisões em uma aplicação executável no navegador. A documentação poderá ser atualizada conforme novas funcionalidades forem implementadas, como o sistema de pontuação, e conforme o desenvolvimento do jogo for concluído.